

Troubleshooting Guide

This guide helps you diagnose and resolve common issues with ReportViewerCore.

Table of Contents

1. [Installation Issues](#)
2. [Report Loading Errors](#)
3. [Rendering Problems](#)
4. [Platform-Specific Issues](#)
5. [Performance Issues](#)
6. [Data Binding Errors](#)
7. [Security and Permissions](#)

Installation Issues

Package Installation Fails

Problem: Cannot install Microsoft.Reporting.NETCore package

Solutions:

1. Check target framework compatibility:

```
<TargetFramework>net8.0</TargetFramework>  
<!-- or net6.0, net7.0, netcoreapp3.1 -->
```

2. Clear NuGet cache:

```
dotnet nuget locals all --clear  
dotnet restore
```

3. Check package source:

```
dotnet nuget list source
```

Missing Dependencies

Problem: Runtime errors about missing assemblies

Solution: Ensure all dependencies are restored:

```
dotnet restore
dotnet build
```

For WinForms projects, ensure you have:

```
<UseWindowsForms>true</UseWindowsForms>
```

Report Loading Errors

FileNotFoundException: Report not found

Problem:

```
FileNotFoundException: Could not find file 'Reports/Invoice.rdlc'
```

Solutions:

1. **Check file path** - Use absolute paths or verify relative path:

```
// Relative to working directory
var reportPath = Path.Combine(Directory.GetCurrentDirectory(), "Reports",
    "Invoice.rdlc");

// Relative to assembly
var reportPath = Path.Combine(AppDomain.CurrentDomain.BaseDirectory, "Reports",
    "Invoice.rdlc");
```

2. **Verify Build Action:**

- File Properties → Build Action → "Content" or "Embedded Resource"
- If "Content", ensure "Copy to Output Directory" → "Copy if newer"

3. **Use embedded resources:**

```
using var stream = Assembly.GetExecutingAssembly()
    .GetManifestResourceStream("YourNamespace.Reports.Invoice.rdlc");
report.LoadReportDefinition(stream);
```

Report Definition Invalid

Problem:

LocalProcessingException: The definition of the report is invalid

Solutions:

1. **Validate RDLC file** - Open in Report Designer and fix any errors
2. **Check XML structure:**

```
<!-- RDLC files must start with -->
<?xml version="1.0" encoding="utf-8"?>
<Report
xmlns="http://schemas.microsoft.com/sqlserver/reporting/2016/01/reportdefinition">
```

3. **Regenerate dataset** if schema changed
4. **Check for unsupported features** like spatial types

Embedded Resource Not Found

Problem:

NullReferenceException when loading from embedded resource

Solution:

1. Check resource name:

```
// List all embedded resources
var assembly = Assembly.GetExecutingAssembly();
var resources = assembly.GetManifestResourceNames();
foreach (var resource in resources)
{
    Console.WriteLine(resource);
}
```

2. Resource name format:

```
{DefaultNamespace}.{FolderPath}.{FileName}
Example: MyApp.Reports.Invoice.rdlc
```

3. Set Build Action to "Embedded Resource"

Rendering Problems

PDF Rendering Fails on Linux

Problem:

```
Exception: Unable to load DLL 'gdi32.dll'
```

Solution: Use Wine (see main README) OR use OpenXML formats:

```
// Instead of PDF
var excel = report.Render("EXCELOPENXML");
var word = report.Render("WORDOPENXML");
```

Fonts Missing in PDF

Problem: PDF displays wrong fonts or boxes instead of characters

Solutions:

1. **Windows:** Install required fonts

2. **Linux:**

```
# Install Microsoft fonts
sudo apt-get install ttf-mscorefonts-installer

# Or use Liberation fonts
sudo apt-get install fonts-liberation
```

3. **Embed fonts** in device info:

```
var deviceInfo = @"
    <DeviceInfo>
        <EmbedFonts>True</EmbedFonts>
    </DeviceInfo>";
report.Render("PDF", deviceInfo);
```

Images Not Appearing

Problem: Images don't show in rendered report

Solutions:

1. Enable external images:

```
report.EnableExternalImages = true;
```

2. Check image source in RDLC:

- Database: Use byte array field
- External: Verify URL is accessible
- Embedded: Ensure image is embedded in assembly

3. Verify image data for database images:

```
// Image field must be byte[]  
public byte[] ProductImage { get; set; }
```

HTML Rendering Issues

Problem: HTML output is broken or incomplete

Solutions:

1. Use HTML5 instead of HTML4.0:

```
var html = report.Render("HTML5");
```

2. Check device info for HTML options:

```
var deviceInfo = @"  
    <DeviceInfo>  
        <Toolbar>False</Toolbar>  
        <StyleStream>True</StyleStream>  
    </DeviceInfo>";
```

3. Images in HTML: HTML5 renderer embeds images as data URLs automatically

Platform-Specific Issues

Blazor WebAssembly

Problem: Report generation fails in Blazor WASM

Solution:

✗ **Cannot run in WASM** - reports require server-side processing

☒ Use backend API:

```
// API Controller
[HttpGet("report/{id}")]
public IActionResult GetReport(int id)
{
    // Generate report on server
    var report = GenerateReport(id);
    return File(report, "application/pdf");
}

// Blazor component calls API
var pdf = await Http.GetByteArrayAsync($"api/report/{id}");
```

MAUI Android

Problem: Report generation crashes on Android

Solutions:

1. **Check permissions** in AndroidManifest.xml:

```
<uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE" />
<uses-permission android:name="android.permission.READ_EXTERNAL_STORAGE" />
```

2. **Request runtime permissions:**

```
var status = await Permissions.RequestAsync<Permissions.StorageWrite>();
```

3. **Use app-specific storage:**

```
var path = Path.Combine(FileSystem.AppDataDirectory, "report.pdf");
```

MAUI iOS

Problem: Report fails with security exception

Solution: Enable required capabilities in Info.plist:

```
<key>NSPhotoLibraryAddUsageDescription</key>
<string>Save reports to photo library</string>
```

macOS

Problem: PDF rendering fails

Solution: Use OpenXML formats or implement Wine setup similar to Linux

Performance Issues

Slow Report Generation

Problem: Reports take too long to generate

Solutions:

1. Profile data loading:

```
var sw = Stopwatch.StartNew();
var data = GetReportData(); // Measure this
Console.WriteLine($"Data loaded in {sw.ElapsedMilliseconds}ms");

sw.Restart();
var report = GenerateReport(data); // And this
Console.WriteLine($"Report generated in {sw.ElapsedMilliseconds}ms");
```

2. Optimize data queries:

- Only fetch required fields
- Filter data at database level
- Use pagination for large datasets

3. Cache report definitions:

```
private static readonly ConcurrentDictionary<string, byte[]> _reportCache =
new();

public byte[] GetReportDefinition(string name)
{
    return _reportCache.GetOrAdd(name, key =>
File.ReadAllBytes($"Reports/{key}.rdlc"));
}
```

4. Use appropriate page count mode:

```
// Faster estimation
report.Render("PDF", null, PageCountMode.Estimate, ...);
```

```
// Accurate but slower
report.Render("PDF", null, PageCountMode.Actual, ...);
```

Memory Issues

Problem: High memory usage or OutOfMemoryException

Solutions:

1. Dispose reports properly:

```
using var report = new LocalReport();
// ... use report
// Automatically disposed
```

2. Stream large reports:

```
public Stream StreamReport()
{
    var stream = new MemoryStream();
    using var report = new LocalReport();

    report.Render("PDF", null, (name, ext, encoding, mime, willSeek) => stream,
        out Warning[] warnings);

    stream.Position = 0;
    return stream;
}
```

3. Limit data size:

```
// Paginate or filter data
var data = GetReportData()
    .Where(d => d.Date >= startDate && d.Date <= endDate)
    .Take(10000) // Limit records
    .ToList();
```

4. Process in batches for multiple reports

Timeout Issues

Problem: Report generation times out in web application

Solutions:

1. Increase timeout:

```
// ASP.NET Core
builder.Services.Configure<FormOptions>(options =>
{
    options.MultipartBodyLengthLimit = 104857600; // 100MB
});

// Kestrel
builder.WebHost.ConfigureKestrel(options =>
{
    options.Limits.KeepAliveTimeout = TimeSpan.FromMinutes(2);
    options.Limits.RequestHeadersTimeout = TimeSpan.FromMinutes(2);
});
```

2. Use background processing:

```
// Queue report for background generation
services.AddHostedService<ReportGenerationService>();
```

3. Implement async generation:

```
public async Task<byte[]> GenerateReportAsync(...)
{
    return await Task.Run(() => {
        using var report = new LocalReport();
        // ... generate report
        return report.Render("PDF");
    });
}
```

Data Binding Errors

Dataset Name Mismatch

Problem:

```
LocalProcessingException: The dataset 'DataSet1' is not found
```

Solutions:

1. **Check dataset name** in RDLC matches code:

```
// RDLC dataset name: "InvoiceItems"
report.DataSources.Add(new ReportDataSource("InvoiceItems", data));
```

2. **View dataset names** in Report Designer:

- Report Data panel → Datasets

3. **Case sensitivity:** Names are case-sensitive

Field Not Found

Problem:

The field 'PropertyName' is not found in the dataset

Solutions:

1. **Verify property names** match exactly:

```
public class InvoiceItem
{
    public string Description { get; set; } // Must match field in RDLC
    public decimal Price { get; set; }
}
```

2. **Refresh dataset** in Report Designer:

- Right-click dataset → Dataset Properties → Refresh Fields

3. **Check data type compatibility**

Null Reference in Expression

Problem:

Warning: The expression contains undefined field reference

Solutions:

1. **Handle nulls in expressions:**

```
=IIF(IsNothing(Fields!Value.Value), "N/A", Fields!Value.Value)
```

2. **Provide default values** in data:

```
var data = items.Select(i => new {  
    Description = i.Description ?? "No description",  
    Price = i.Price ?? 0m  
});
```

Security and Permissions

Security Exception

Problem:

```
SecurityException: Request failed with security exception
```

Solutions:

1. **Enable hyperlinks** if needed:

```
report.EnableHyperlinks = true;
```

2. **Enable external images** (carefully):

```
report.EnableExternalImages = true;
```

3. **Check file permissions:**

```
# Linux  
chmod 644 Reports/*.rdlc
```

Authentication Failed (Server Reports)

Problem: Cannot connect to Reporting Services

Solutions:

1. **Check credentials:**

```
report.ReportServerCredentials = new ReportServerCredentials  
{
```

```
NetworkCredentials = new NetworkCredential("user", "pass", "DOMAIN")
};
```

2. Verify server URL:

```
report.ReportServerUrl = new Uri("http://server/ReportServer");
// NOT: http://server/Reports (that's the web portal)
```

3. Test connection separately:

```
curl -u "DOMAIN\user:pass" http://server/ReportServer
```

Common Error Messages

"The report definition is not valid"

Causes:

- Corrupt RDLC file
- Unsupported features (spatial types)
- XML syntax errors

Fix: Validate RDLC in Report Designer

"Cannot find compilation library for Roslyn"

Cause: Report uses expressions that need compilation

Fix: Ensure Microsoft.CodeAnalysis packages are available (they should be included automatically)

"Report processing exception"

Generic error - Enable detailed logging:

```
// Add logging
using Microsoft.Extensions.Logging;

public class ReportService
{
    private readonly ILogger<ReportService> _logger;

    public byte[] Generate()
    {
        try
        {
            // ... generate report
        }
    }
}
```

```

    }
    catch (LocalProcessingException ex)
    {
        _logger.LogError(ex, "Report processing failed");
        _logger.LogError("Inner exception: {InnerException}",
            ex.InnerException?.Message);
        throw;
    }
}

```

Debugging Tips

Enable Trace Output

```

System.Diagnostics.Trace.Listeners.Add(new ConsoleTraceListener());

using var report = new LocalReport();
// ... Report processing warnings will be written to console

```

Capture All Warnings

```

Warning[] warnings;
var result = report.Render("PDF", null, PageCountMode.Actual,
    out _, out _, out _, out _, out warnings);

foreach (var warning in warnings)
{
    Console.WriteLine($"[{warning.Severity}] {warning.Code}");
    Console.WriteLine($"  Message: {warning.Message}");
    Console.WriteLine($"  Object: {warning.ObjectName}");
    Console.WriteLine($"  Property: {warning.ObjectType}");
}

```

Test with Minimal Report

Create a simple test report:

```

public void TestMinimalReport()
{
    using var report = new LocalReport();

    // Create minimal RDLC programmatically or load simple one
    var data = new[] { new { Value = "Test" } };
}

```

```
report.DataSources.Add(new ReportDataSource("Data", data));

var pdf = report.Render("PDF");

Assert.True(pdf.Length > 0);
}
```

Getting Help

If you're still experiencing issues:

1. **Check GitHub issues:** <https://github.com/lkosson/reportviewercore/issues>
2. **Search Stack Overflow:** Tag with `reportviewer` and `.net-core`
3. **Create minimal reproduction:** Isolate the problem
4. **Include:**
 - .NET version
 - Operating system
 - Full error message and stack trace
 - Minimal code to reproduce issue

Additional Resources

- [Main README](#) - Project overview
- [Getting Started](#) - Basic usage
- [Advanced Topics](#) - Complex scenarios
- [GitHub Repository](#) - Source code and issues